

▼ Titanic Survival Prediction Using Machine Learning and Neural Networks

Importing dependencies

```
#data handling
import pandas as pd
import numpy as np

#data visualization
import matplotlib.pyplot as plt
import seaborn as sns

#data preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

#machine learning models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

#model evaluation
from sklearn.metrics import(
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

#deep learning
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping

#reproducibility
import random
```

Setting random seeds for reproducibility

```
SEED = 42

np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)
```

Dataset Loading

```
df=pd.read_csv('/content/Titanic-Dataset.csv')
```

Information about the dataset

```
print("\nfirst 5 rows:",df.head())
print("\ndataset shape:",df.shape)
print("\nname of columns:",df.columns.tolist())
print("\ndata type:",df.dtypes)
```

```
first 5 rows:      PassengerId  Survived  Pclass \
0                1         0       3
1                2         1       1
2                3         1       3
```

```

3          4          1          1
4          5          0          3

      Name      Sex  Age  SibSp  \
0      Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2      Heikkinen, Miss. Laina    female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4      Allen, Mr. William Henry    male  35.0      0

      Parch      Ticket    Fare Cabin Embarked
0      0          A/5 21171    7.2500   NaN      S
1      0          PC 17599   71.2833   C85      C
2      0  STON/O2. 3101282    7.9250   NaN      S
3      0          113803   53.1000  C123      S
4      0          373450    8.0500   NaN      S

dataset shape: (891, 12)

name of columns: ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embar

data type: PassengerId      int64
Survived      int64
Pclass      int64
Name      object
Sex      object
Age      float64
SibSp      int64
Parch      int64
Ticket      object
Fare      float64
Cabin      object
Embarked      object
dtype: object

```

Checking for missing values

```

missing_percentage = (df.isnull().sum() / len(df)) * 100

missing_info = pd.DataFrame({
    "Missing Values": df.isnull().sum(),
    "Missing Percentage": missing_percentage
})

print(missing_info[missing_info["Missing Values"] > 0])

```

	Missing Values	Missing Percentage
Age	177	19.865320
Cabin	687	77.104377
Embarked	2	0.224467

```

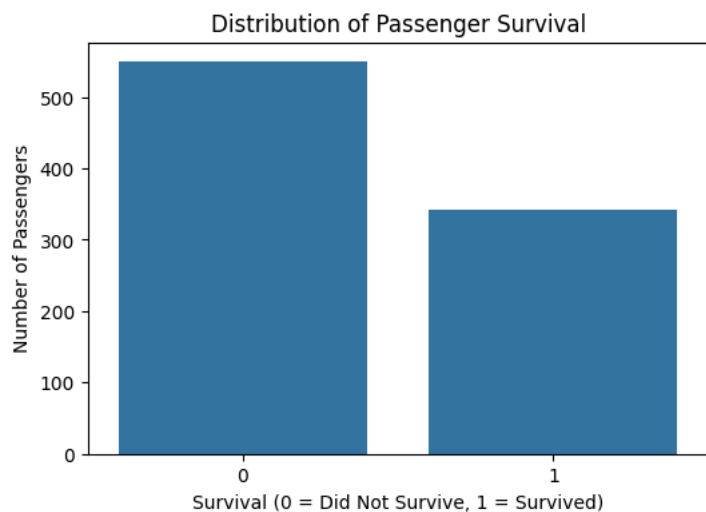
plt.figure(figsize=(6, 4))

sns.countplot(data=df, x="Survived")

plt.title("Distribution of Passenger Survival")
plt.xlabel("Survival (0 = Did Not Survive, 1 = Survived)")
plt.ylabel("Number of Passengers")

plt.show()

```

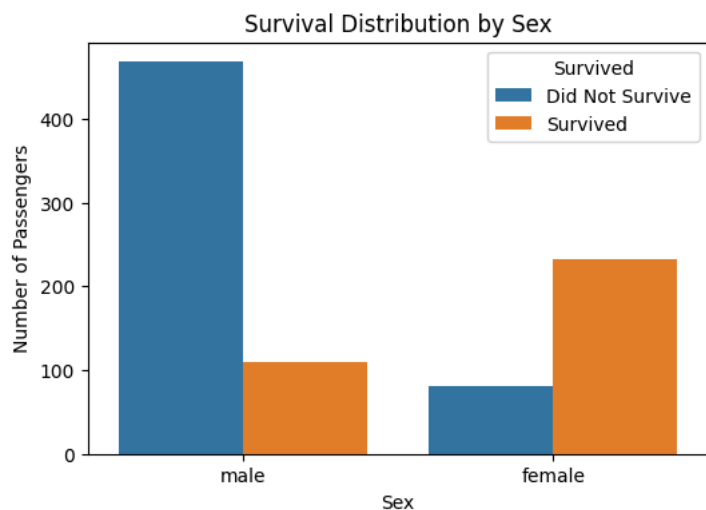


```
plt.figure(figsize=(6, 4))

sns.countplot(data=df, x="Sex", hue="Survived")

plt.title("Survival Distribution by Sex")
plt.xlabel("Sex")
plt.ylabel("Number of Passengers")
plt.legend(
    title="Survived",
    labels=["Did Not Survive", "Survived"]
)

plt.show()
```



Handling missing values

```
# fill missing Age values with the median
df["Age"] = df["Age"].fillna(df["Age"].median())

# fill missing Embarked values with the mode
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Drop Cabin because it has too many missing values
df = df.drop(columns=["Cabin"])

# Verify that missing values have been handled
print(df.isnull().sum())
```

```
PassengerId    0
Survived       0
Pclass         0
```

```
Name      0
Sex       0
Age       0
SibSp     0
Parch     0
Ticket    0
Fare      0
Embarked  0
dtype: int64
```

Feature Selection

```
features = [
    "Pclass",
    "Sex",
    "Age",
    "SibSp",
    "Parch",
    "Fare",
    "Embarked"
]

target = "Survived"
X = df[features]
y = df["Survived"]

print("Features shape:", X.shape)
print("Target shape:", y.shape)

display(X.head())
```

Features shape: (891, 7)
Target shape: (891,)

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	male	22.0	1	0	7.2500	S
1	1	female	38.0	1	0	71.2833	C
2	3	female	26.0	0	0	7.9250	S
3	1	female	35.0	1	0	53.1000	S
4	3	male	35.0	0	0	8.0500	S

Why these features? Pclass - Passenger's socioeconomic/travel class Sex - Likely important for survival Age - Children and adults may have had different survival chances SibSp - Number of siblings/spouses aboard Parch - Number of parents/children aboard Fare - Can indirectly reflect passenger class and socioeconomic status Embarked - Port of embarkation may capture differences in passenger groups

Why are we not using these?

PassengerId - Just an identifier; it has no meaningful relationship with survival. Name - Raw text and high-cardinality; we can later engineer a feature like Title from it as an MSc-level enhancement. Ticket - High-cardinality categorical feature and requires additional feature engineering. Cabin - Already dropped due to 77.1% missing values.

Encode Categorical Features

```
# Define categorical and numerical features

categorical_features = ["Sex", "Embarked"]

numerical_features = [
    "Pclass",
    "Age",
    "SibSp",
    "Parch",
    "Fare"
]

# Create preprocessing transformers

categorical_transformer = OneHotEncoder(
    handle_unknown="ignore",
    drop="first"
```

```

)

numerical_transformer = StandardScaler()
# Combine preprocessing steps

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numerical_transformer, numerical_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

```

Data splitting

```

# Split data into training (70%), validation (15%), and test (15%)

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=SEED,
    stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=SEED,
    stratify=y_temp
)

print("Training set:", X_train.shape)
print("Validation set:", X_val.shape)
print("Test set:", X_test.shape)

```

```

Training set: (623, 7)
Validation set: (134, 7)
Test set: (134, 7)

```

Fit the Preprocessor on Training Data Only

```

# Fit the preprocessor only on the training data

X_train_processed = preprocessor.fit_transform(X_train)

# Use the learned transformations to transform validation and test data
X_val_processed = preprocessor.transform(X_val)
X_test_processed = preprocessor.transform(X_test)

# Check the shapes
print("Processed training shape:", X_train_processed.shape)
print("Processed validation shape:", X_val_processed.shape)
print("Processed test shape:", X_test_processed.shape)

```

```

Processed training shape: (623, 8)
Processed validation shape: (134, 8)
Processed test shape: (134, 8)

```

Baseline Model - Train Logistic Regression

```

# Baseline Model: Logistic Regression

log_reg = LogisticRegression(
    random_state=SEED,
    max_iter=1000
)

log_reg.fit(X_train_processed, y_train)

```

```

LogisticRegression
LogisticRegression(max_iter=1000, random_state=42)

```

Evaluating the Logistic Regression Baseline on the Validation Set

```
# Make predictions on the validation set

y_val_pred_log = log_reg.predict(X_val_processed)

# Calculate evaluation metrics

log_val_accuracy = accuracy_score(y_val, y_val_pred_log)
log_val_precision = precision_score(y_val, y_val_pred_log)
log_val_recall = recall_score(y_val, y_val_pred_log)
log_val_f1 = f1_score(y_val, y_val_pred_log)

print("Logistic Regression - Validation Results")
print(f"Accuracy: {log_val_accuracy:.4f}")
print(f"Precision: {log_val_precision:.4f}")
print(f"Recall: {log_val_recall:.4f}")
print(f"F1 Score: {log_val_f1:.4f}")
```

```
Logistic Regression - Validation Results
Accuracy: 0.8582
Precision: 0.8511
Recall: 0.7692
F1 Score: 0.8081
```

Logistic Regression Confusion Matrix

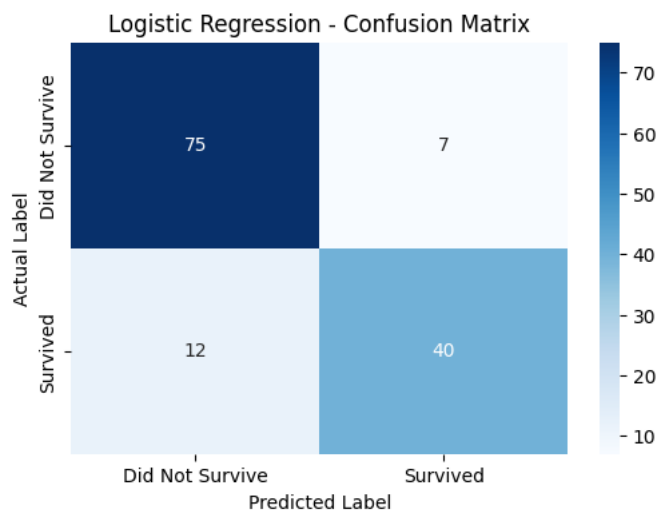
```
# Confusion Matrix for Logistic Regression

cm_log = confusion_matrix(y_val, y_val_pred_log)

plt.figure(figsize=(6, 4))

sns.heatmap(
    cm_log,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Did Not Survive", "Survived"],
    yticklabels=["Did Not Survive", "Survived"]
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Logistic Regression - Confusion Matrix")
plt.show()
```



Build Neural Network Model

```
mlp_model = keras.Sequential([
```

```

layers.Input(shape=(X_train_processed.shape[1],)),

layers.Dense(32, activation="relu"),

layers.Dense(16, activation="relu"),

layers.Dense(1, activation="sigmoid")
])

# Display the model architecture
mlp_model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	288
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 1)	17

Total params: 833 (3.25 KB)
 Trainable params: 833 (3.25 KB)
 Non-trainable params: 0 (0.00 B)

Compiling Neural Network

```

mlp_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

```

Training Neural Network

```

# Define Early Stopping
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)

# Train the Neural Network

history = mlp_model.fit(
    X_train_processed,
    y_train,
    validation_data=(X_val_processed, y_val),
    epochs=100,
    batch_size=32,
    callbacks=[early_stopping],
    verbose=1
)

```

```

Epoch 33/100
20/20 ————— 0s 11ms/step - accuracy: 0.8523 - loss: 0.3727 - val_accuracy: 0.8731 - val_loss: 0.3682
Epoch 34/100
20/20 ————— 0s 10ms/step - accuracy: 0.8523 - loss: 0.3717 - val_accuracy: 0.8731 - val_loss: 0.3683
Epoch 35/100
20/20 ————— 0s 10ms/step - accuracy: 0.8539 - loss: 0.3708 - val_accuracy: 0.8731 - val_loss: 0.3680
Epoch 36/100
20/20 ————— 0s 12ms/step - accuracy: 0.8539 - loss: 0.3698 - val_accuracy: 0.8731 - val_loss: 0.3680
Epoch 37/100
20/20 ————— 0s 17ms/step - accuracy: 0.8539 - loss: 0.3689 - val_accuracy: 0.8731 - val_loss: 0.3679
Epoch 38/100
20/20 ————— 0s 11ms/step - accuracy: 0.8571 - loss: 0.3680 - val_accuracy: 0.8731 - val_loss: 0.3679
Epoch 39/100
20/20 ————— 0s 12ms/step - accuracy: 0.8571 - loss: 0.3672 - val_accuracy: 0.8731 - val_loss: 0.3681
Epoch 40/100
20/20 ————— 0s 11ms/step - accuracy: 0.8571 - loss: 0.3663 - val_accuracy: 0.8731 - val_loss: 0.3678
Epoch 41/100
20/20 ————— 0s 6ms/step - accuracy: 0.8571 - loss: 0.3655 - val_accuracy: 0.8731 - val_loss: 0.3682
Epoch 42/100
20/20 ————— 0s 7ms/step - accuracy: 0.8571 - loss: 0.3647 - val_accuracy: 0.8731 - val_loss: 0.3680
Epoch 43/100
20/20 ————— 0s 6ms/step - accuracy: 0.8571 - loss: 0.3640 - val_accuracy: 0.8731 - val_loss: 0.3682
Epoch 44/100
20/20 ————— 0s 7ms/step - accuracy: 0.8571 - loss: 0.3631 - val_accuracy: 0.8657 - val_loss: 0.3683
Epoch 45/100
20/20 ————— 0s 6ms/step - accuracy: 0.8555 - loss: 0.3624 - val_accuracy: 0.8657 - val_loss: 0.3686
Epoch 46/100
20/20 ————— 0s 6ms/step - accuracy: 0.8555 - loss: 0.3616 - val_accuracy: 0.8657 - val_loss: 0.3686
Epoch 47/100
20/20 ————— 0s 7ms/step - accuracy: 0.8555 - loss: 0.3609 - val_accuracy: 0.8657 - val_loss: 0.3687
Epoch 48/100
20/20 ————— 0s 6ms/step - accuracy: 0.8555 - loss: 0.3602 - val_accuracy: 0.8657 - val_loss: 0.3690
Epoch 49/100
20/20 ————— 0s 6ms/step - accuracy: 0.8555 - loss: 0.3595 - val_accuracy: 0.8657 - val_loss: 0.3694
Epoch 50/100
20/20 ————— 0s 6ms/step - accuracy: 0.8555 - loss: 0.3588 - val_accuracy: 0.8657 - val_loss: 0.3695

```

Evaluate MLP on the validation set

```

# Predict survival probabilities on the validation set

y_val_prob_mlp = mlp_model.predict(X_val_processed)

# Convert probabilities into binary predictions
y_val_pred_mlp = (y_val_prob_mlp >= 0.5).astype(int).flatten()

# Calculate evaluation metrics

mlp_val_accuracy = accuracy_score(y_val, y_val_pred_mlp)
mlp_val_precision = precision_score(y_val, y_val_pred_mlp)
mlp_val_recall = recall_score(y_val, y_val_pred_mlp)
mlp_val_f1 = f1_score(y_val, y_val_pred_mlp)

print("MLP Neural Network - Validation Results")
print(f"Accuracy: {mlp_val_accuracy:.4f}")
print(f"Precision: {mlp_val_precision:.4f}")
print(f"Recall: {mlp_val_recall:.4f}")
print(f"F1 Score: {mlp_val_f1:.4f}")

5/5 ————— 0s 16ms/step
MLP Neural Network - Validation Results
Accuracy: 0.8731
Precision: 0.9268
Recall: 0.7308
F1 Score: 0.8172

```

Compare Logistic Regression and MLP on the validation set

```

comparison_df = pd.DataFrame({
    "Model": ["Logistic Regression", "MLP Neural Network"],
    "Accuracy": [log_val_accuracy, mlp_val_accuracy],
    "Precision": [log_val_precision, mlp_val_precision],
    "Recall": [log_val_recall, mlp_val_recall],
    "F1 Score": [log_val_f1, mlp_val_f1]
})

comparison_df

```


	Model	Accuracy	Precision	Recall	F1 Score	
0	Logistic Regression	0.858209	0.851064	0.769231	0.808081	
1	MLP Neural Network	0.873134	0.926829	0.730769	0.817204	

Next steps: [Generate code with comparison_df](#) [New interactive sheet](#)

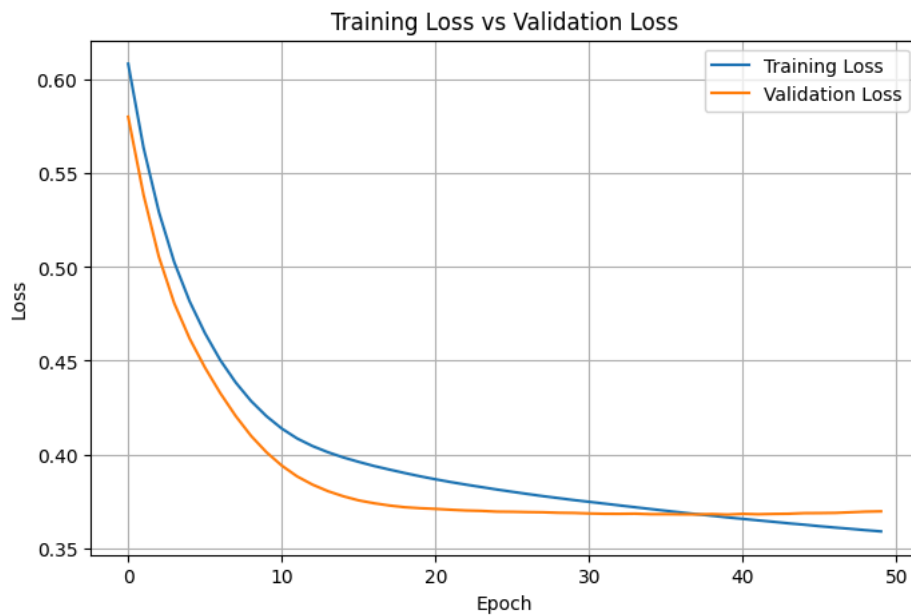
Plot Training and Validation Curves

```
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss vs Validation Loss")
plt.legend()
plt.grid(True)

plt.show()
```



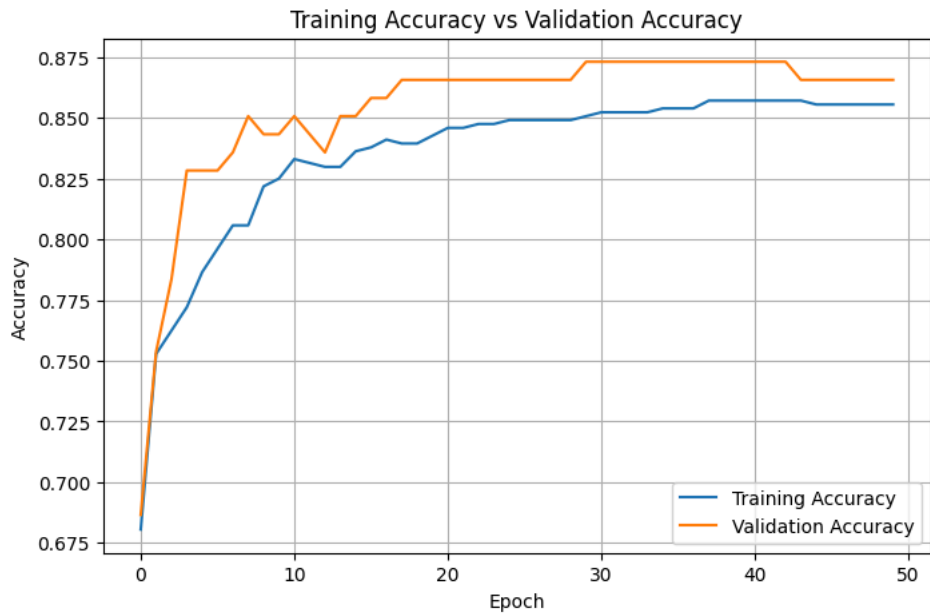
Plot Training and Validation Accuracy

```
plt.figure(figsize=(8, 5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training Accuracy vs Validation Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



The model learned effectively, as both training and validation loss decreased significantly during training. The validation loss reached a plateau around the later epochs and showed only minor fluctuations, while the training and validation accuracy remained relatively close. Therefore, the model does not show signs of severe overfitting or underfitting. Early stopping helped prevent unnecessary training after validation performance stopped improving.

Building Model drop out

```
# Experiment: MLP with Dropout

dropout_model = keras.Sequential([

    layers.Input(shape=(X_train_processed.shape[1],)),

    layers.Dense(32, activation="relu"),
    layers.Dropout(0.2),

    layers.Dense(16, activation="relu"),
    layers.Dropout(0.1),

    layers.Dense(1, activation="sigmoid")
])

dropout_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

dropout_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 32)	288
dropout (Dropout)	(None, 32)	0
dense_4 (Dense)	(None, 16)	528
dropout_1 (Dropout)	(None, 16)	0
dense_5 (Dense)	(None, 1)	17

Total params: 833 (3.25 KB)
Trainable params: 833 (3.25 KB)

Train Dropout Model

```
# Early stopping for the Dropout model
```

```
early_stopping_dropout = EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)
```

```
# Train the Dropout model
```

```
history_dropout = dropout_model.fit(
    X_train_processed,
    y_train,
    validation_data=(X_val_processed, y_val),
    epochs=100,
    batch_size=32,
    callbacks=[early_stopping_dropout],
    verbose=1
)
```

```
Epoch 72/100
20/20 ————— 0s 7ms/step - accuracy: 0.8283 - loss: 0.4051 - val_accuracy: 0.8731 - val_loss: 0.3658
Epoch 73/100
20/20 ————— 0s 7ms/step - accuracy: 0.8379 - loss: 0.3984 - val_accuracy: 0.8731 - val_loss: 0.3665
Epoch 74/100
20/20 ————— 0s 6ms/step - accuracy: 0.8523 - loss: 0.3828 - val_accuracy: 0.8731 - val_loss: 0.3655
Epoch 75/100
20/20 ————— 0s 6ms/step - accuracy: 0.8331 - loss: 0.3993 - val_accuracy: 0.8731 - val_loss: 0.3664
Epoch 76/100
20/20 ————— 0s 7ms/step - accuracy: 0.8363 - loss: 0.3967 - val_accuracy: 0.8731 - val_loss: 0.3661
Epoch 77/100
20/20 ————— 0s 6ms/step - accuracy: 0.8459 - loss: 0.3933 - val_accuracy: 0.8731 - val_loss: 0.3644
Epoch 78/100
20/20 ————— 0s 6ms/step - accuracy: 0.8459 - loss: 0.3758 - val_accuracy: 0.8731 - val_loss: 0.3640
Epoch 79/100
20/20 ————— 0s 7ms/step - accuracy: 0.8427 - loss: 0.3870 - val_accuracy: 0.8731 - val_loss: 0.3638
Epoch 80/100
20/20 ————— 0s 6ms/step - accuracy: 0.8315 - loss: 0.3888 - val_accuracy: 0.8731 - val_loss: 0.3632
Epoch 81/100
20/20 ————— 0s 6ms/step - accuracy: 0.8395 - loss: 0.3956 - val_accuracy: 0.8731 - val_loss: 0.3624
Epoch 82/100
20/20 ————— 0s 6ms/step - accuracy: 0.8411 - loss: 0.3785 - val_accuracy: 0.8731 - val_loss: 0.3614
Epoch 83/100
20/20 ————— 0s 7ms/step - accuracy: 0.8395 - loss: 0.3881 - val_accuracy: 0.8731 - val_loss: 0.3608
Epoch 84/100
20/20 ————— 0s 7ms/step - accuracy: 0.8347 - loss: 0.3986 - val_accuracy: 0.8731 - val_loss: 0.3605
Epoch 85/100
20/20 ————— 0s 7ms/step - accuracy: 0.8507 - loss: 0.3885 - val_accuracy: 0.8731 - val_loss: 0.3601
Epoch 86/100
20/20 ————— 0s 6ms/step - accuracy: 0.8427 - loss: 0.3976 - val_accuracy: 0.8731 - val_loss: 0.3612
Epoch 87/100
20/20 ————— 0s 6ms/step - accuracy: 0.8379 - loss: 0.3905 - val_accuracy: 0.8731 - val_loss: 0.3622
Epoch 88/100
20/20 ————— 0s 6ms/step - accuracy: 0.8411 - loss: 0.4002 - val_accuracy: 0.8731 - val_loss: 0.3622
Epoch 89/100
20/20 ————— 0s 6ms/step - accuracy: 0.8491 - loss: 0.3847 - val_accuracy: 0.8731 - val_loss: 0.3609
Epoch 90/100
20/20 ————— 0s 7ms/step - accuracy: 0.8523 - loss: 0.3869 - val_accuracy: 0.8657 - val_loss: 0.3608
Epoch 91/100
20/20 ————— 0s 7ms/step - accuracy: 0.8475 - loss: 0.3875 - val_accuracy: 0.8657 - val_loss: 0.3615
Epoch 92/100
20/20 ————— 0s 8ms/step - accuracy: 0.8395 - loss: 0.3800 - val_accuracy: 0.8657 - val_loss: 0.3608
Epoch 93/100
20/20 ————— 0s 6ms/step - accuracy: 0.8507 - loss: 0.3847 - val_accuracy: 0.8731 - val_loss: 0.3600
Epoch 94/100
20/20 ————— 0s 6ms/step - accuracy: 0.8459 - loss: 0.3904 - val_accuracy: 0.8731 - val_loss: 0.3598
Epoch 95/100
20/20 ————— 0s 6ms/step - accuracy: 0.8395 - loss: 0.3922 - val_accuracy: 0.8657 - val_loss: 0.3604
Epoch 96/100
20/20 ————— 0s 7ms/step - accuracy: 0.8443 - loss: 0.3896 - val_accuracy: 0.8657 - val_loss: 0.3616
Epoch 97/100
20/20 ————— 0s 7ms/step - accuracy: 0.8459 - loss: 0.3800 - val_accuracy: 0.8657 - val_loss: 0.3621
Epoch 98/100
20/20 ————— 0s 6ms/step - accuracy: 0.8507 - loss: 0.3772 - val_accuracy: 0.8657 - val_loss: 0.3630
Epoch 99/100
20/20 ————— 0s 7ms/step - accuracy: 0.8379 - loss: 0.3964 - val_accuracy: 0.8657 - val_loss: 0.3635
Epoch 100/100
20/20 ————— 0s 6ms/step - accuracy: 0.8427 - loss: 0.3723 - val_accuracy: 0.8657 - val_loss: 0.3631
```

Evaluate dropout model

```
# Predict survival probabilities using the Dropout model

y_val_prob_dropout = dropout_model.predict(X_val_processed)

# Convert probabilities into binary predictions
y_val_pred_dropout = (y_val_prob_dropout >= 0.5).astype(int).flatten()

# Calculate evaluation metrics

dropout_val_accuracy = accuracy_score(y_val, y_val_pred_dropout)
dropout_val_precision = precision_score(y_val, y_val_pred_dropout)
dropout_val_recall = recall_score(y_val, y_val_pred_dropout)
dropout_val_f1 = f1_score(y_val, y_val_pred_dropout)

print("MLP with Dropout - Validation Results")
print(f"Accuracy: {dropout_val_accuracy:.4f}")
print(f"Precision: {dropout_val_precision:.4f}")
print(f"Recall: {dropout_val_recall:.4f}")
print(f"F1 Score: {dropout_val_f1:.4f}")
```

5/5 0s 16ms/step

MLP with Dropout - Validation Results

Accuracy: 0.8731

Precision: 0.9268

Recall: 0.7308

F1 Score: 0.8172

Final Validation Comparison Table

```
# Final validation comparison of all models

final_comparison = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Original MLP",
        "MLP with Dropout"
    ],
    "Accuracy": [
        log_val_accuracy,
        mlp_val_accuracy,
        dropout_val_accuracy
    ],
    "Precision": [
        log_val_precision,
        mlp_val_precision,
        dropout_val_precision
    ],
    "Recall": [
        log_val_recall,
        mlp_val_recall,
        dropout_val_recall
    ],
    "F1 Score": [
        log_val_f1,
        mlp_val_f1,
        dropout_val_f1
    ]
})

final_comparison
```

	Model	Accuracy	Precision	Recall	F1 Score	
0	Logistic Regression	0.858209	0.851064	0.769231	0.808081	
1	Original MLP	0.873134	0.926829	0.730769	0.817204	
2	MLP with Dropout	0.873134	0.926829	0.730769	0.817204	

Next steps:

[Generate code with final_comparison](#)

[New interactive sheet](#)

Final Evaluation on the Test Set

```
# Make predictions on the unseen test set

y_test_prob_mlp = mlp_model.predict(X_test_processed)

# Convert probabilities into binary predictions
y_test_pred_mlp = (y_test_prob_mlp >= 0.5).astype(int).flatten()

# Calculate final test metrics

test_accuracy = accuracy_score(y_test, y_test_pred_mlp)
test_precision = precision_score(y_test, y_test_pred_mlp)
test_recall = recall_score(y_test, y_test_pred_mlp)
test_f1 = f1_score(y_test, y_test_pred_mlp)

print("Final MLP - Test Set Results")
print(f"Accuracy: {test_accuracy:.4f}")
print(f"Precision: {test_precision:.4f}")
print(f"Recall: {test_recall:.4f}")
print(f"F1 Score: {test_f1:.4f}")
```

```
5/5 ————— 0s 7ms/step
Final MLP - Test Set Results
Accuracy: 0.7836
Precision: 0.7750
Recall: 0.6078
F1 Score: 0.6813
```

Final Test Confusion Matrix

```
# Confusion Matrix for the final MLP on the test set

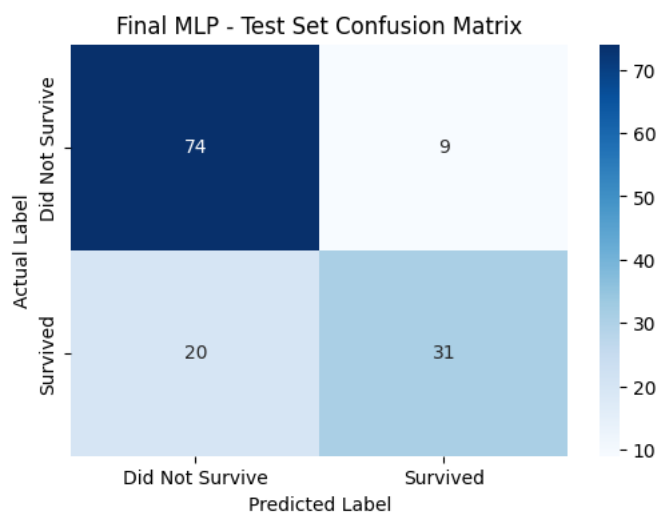
cm_test = confusion_matrix(y_test, y_test_pred_mlp)

plt.figure(figsize=(6, 4))

sns.heatmap(
    cm_test,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Did Not Survive", "Survived"],
    yticklabels=["Did Not Survive", "Survived"]
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Final MLP - Test Set Confusion Matrix")

plt.show()
```



```
# Detailed classification report

print(classification_report(
    y_test,
```

```
y_test_pred_mlp,  
target_names=["Did Not Survive", "Survived"]  
)
```

	precision	recall	f1-score	support
Did Not Survive	0.79	0.89	0.84	83
Survived	0.78	0.61	0.68	51
accuracy			0.78	134
macro avg	0.78	0.75	0.76	134
weighted avg	0.78	0.78	0.78	134

Final Conclusion

This project developed and evaluated a Feed Forward Neural Network (MLP) to predict passenger survival using the Titanic dataset. The dataset was preprocessed by handling missing values, removing the Cabin feature due to its high percentage of missing data, encoding categorical variables, and scaling numerical features. The data was divided into training, validation, and test sets to ensure a fair evaluation.

A Logistic Regression model was first used as a baseline. The baseline achieved a validation accuracy of 85.82%. The MLP improved the validation accuracy to 87.31% and also achieved higher precision and F1-score. An additional experiment was conducted by adding